

Supplementary Materials

Benchmarking deep and hybrid quantum-classical models for Gallo-Roman ceramic sherd classification: a reproducible evaluation framework

Cyrille Chaidron^{1*}, Hafsa Taiebi Imrani²

¹ UMR 7041 ArScAn & U.R. 4284 TrAme, University of Picardie Jules Verne, Amiens, France ² Faculty of Science, Ibn Tofail University, Kenitra, Morocco

*Corresponding author: Cyrille Chaidron — cyrille@arteka.tech

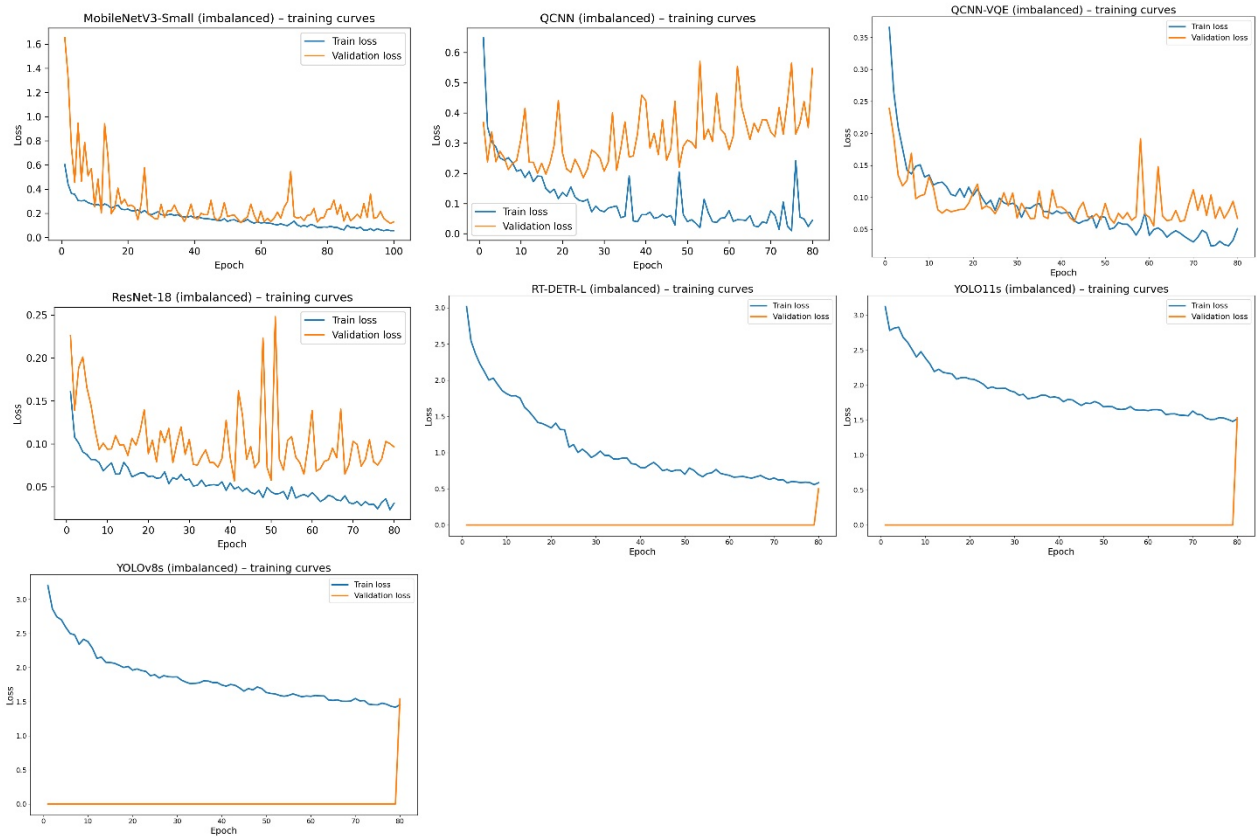


Figure S1 | Training and validation loss curves for the imbalanced scenario (seed 42).

Training and validation loss evolution across epochs for all evaluated models trained on the imbalanced dataset: MobileNetV3-Small, ResNet-18, QCNN, and QCNN-VQE (classification on ground-truth crops), as well as YOLOv8-s, YOLOv11-s, and RT-DETR-L (end-to-end detection on full images).

For crop-based classifiers, losses show stable convergence without pronounced overfitting despite class imbalance. For detection models, validation loss is reported only at evaluation

checkpoints defined by the Ultralytics framework, explaining the sparse or zero-valued validation curves. Best-performing checkpoints reported in the main text are selected based on validation criteria, as described in the Methods section.

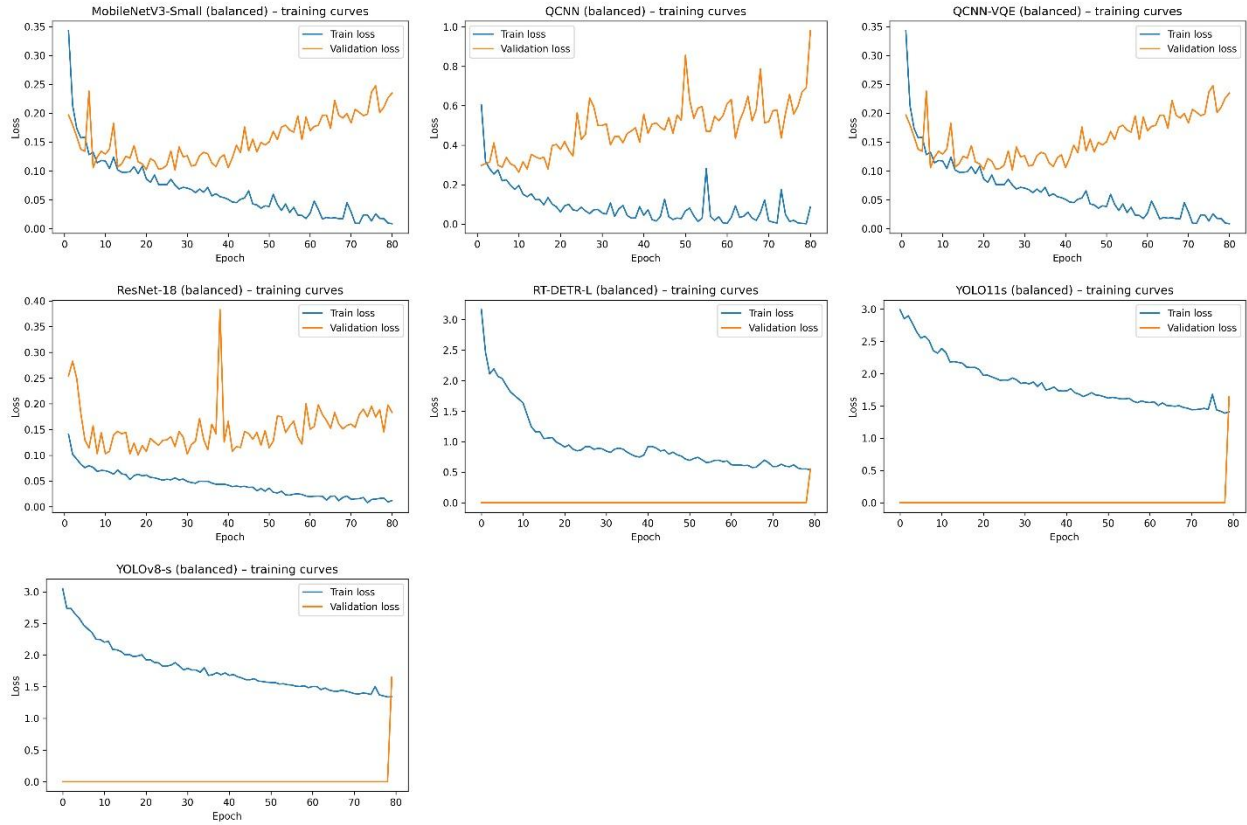


Figure S2 |Training and validation loss curves for the balanced scenario (seed 42).

Training and validation loss evolution across epochs for all evaluated models trained on the balanced dataset: MobileNetV3-Small, ResNet-18, QCNN, and QCNN-VQE (classification on ground-truth crops), as well as YOLOv8-s, YOLOv11-s, and RT-DETR-L (end-to-end detection on full images).

For crop-based classifiers, losses indicate stable convergence after rebalancing, with no systematic overfitting. For detection models, validation loss is reported only at evaluation checkpoints defined by the Ultralytics framework, resulting in sparse or delayed validation curves. Best checkpoints used in the main analysis are selected based on validation criteria described in the Methods section.

This section provides representative learning curves for the main seed-42 runs in both balanced and imbalanced settings. These curves document convergence behaviour and training stability. For classification models, both loss and validation performance are reported. For detection models, only training classification loss is shown, because intermediate validation metrics were not computed densely enough during training to yield informative epoch-by-epoch curves.

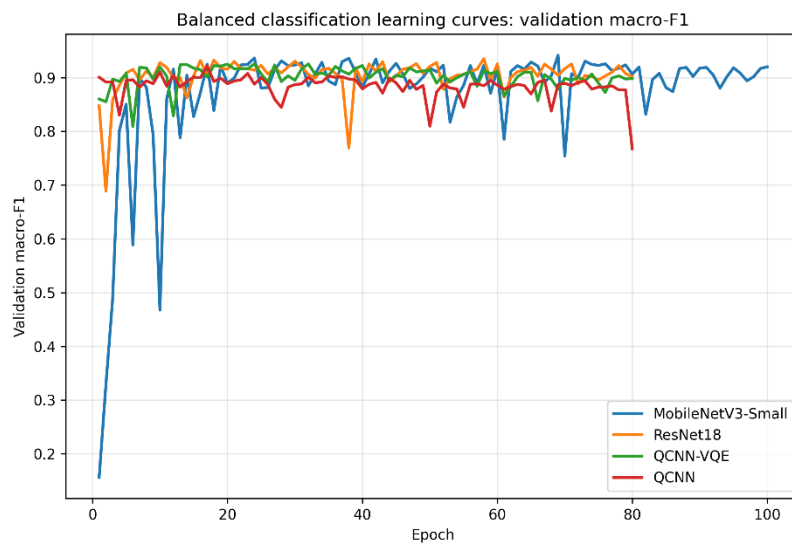
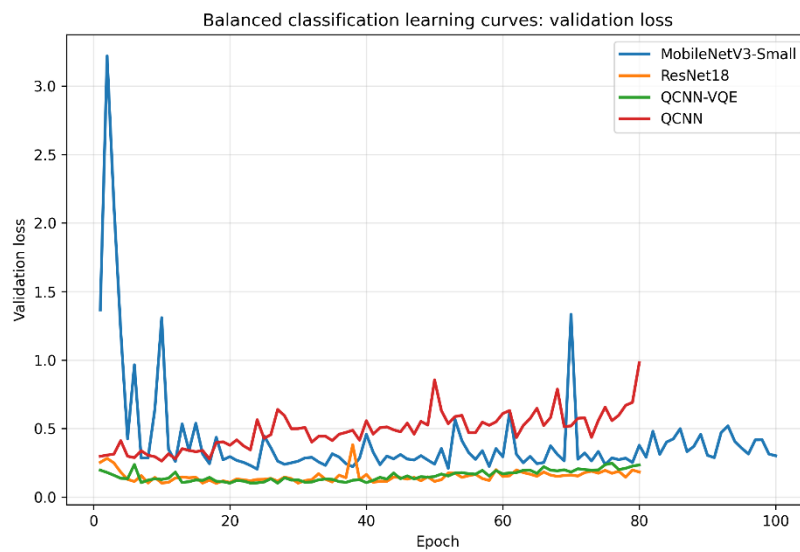
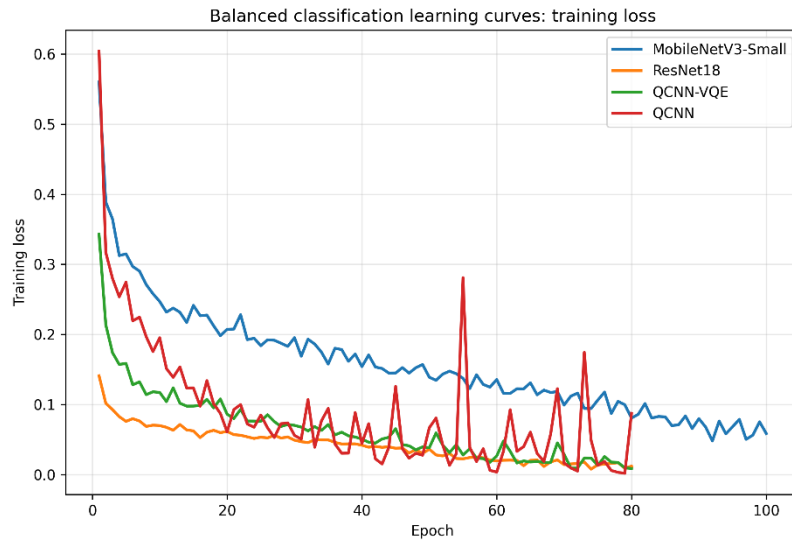


Figure S3 | Representative learning curves (seed 42) for the balanced classification scenario: training loss, validation loss, and validation macro-F1 for MobileNetV3-Small, ResNet-18, QCNN-VQE, and QCNN

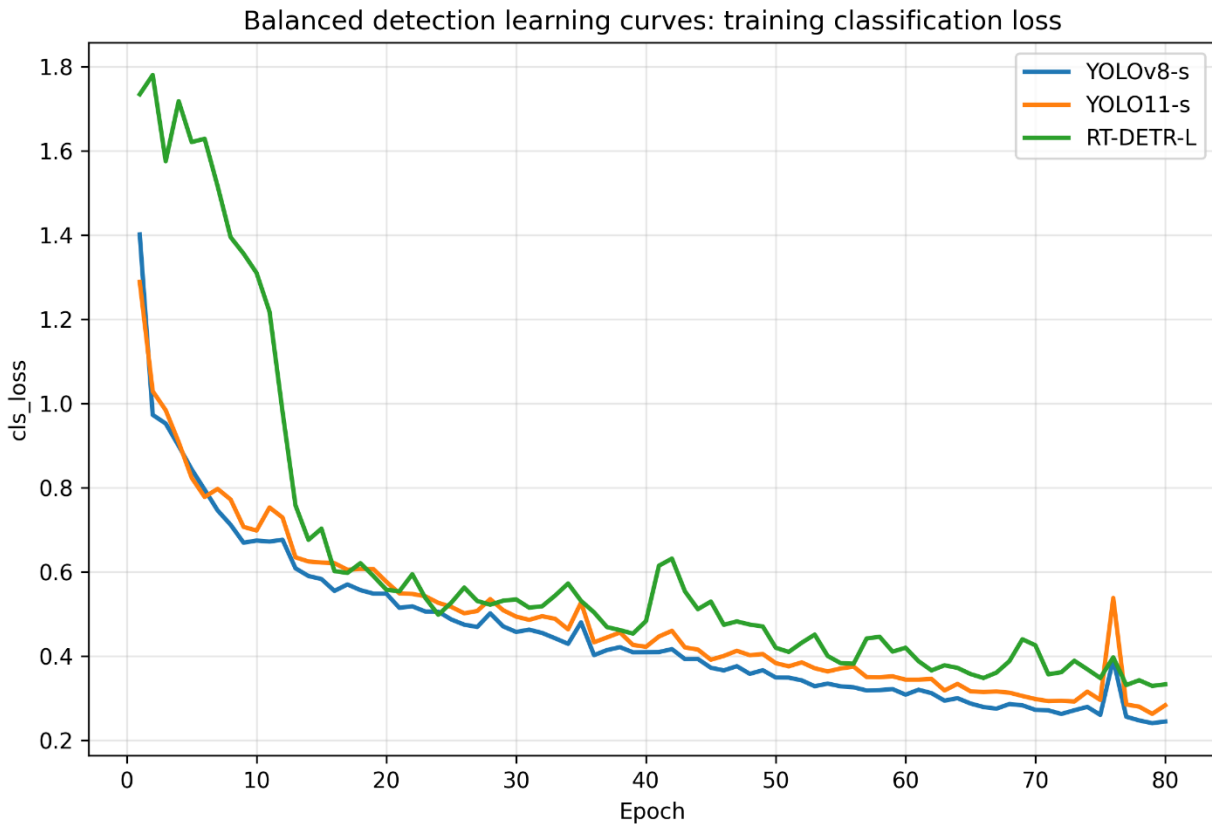


Figure S4 | Representative learning curves (seed 42) for the balanced detection scenario. Shown here is the training classification loss (train/cls_loss) for YOLOv8-s, YOLOv11-s, and RT-DETR-L. Validation mAP and precision/recall were not computed at every epoch in the present Ultralytics configuration, so intermediate validation curves are not informative.

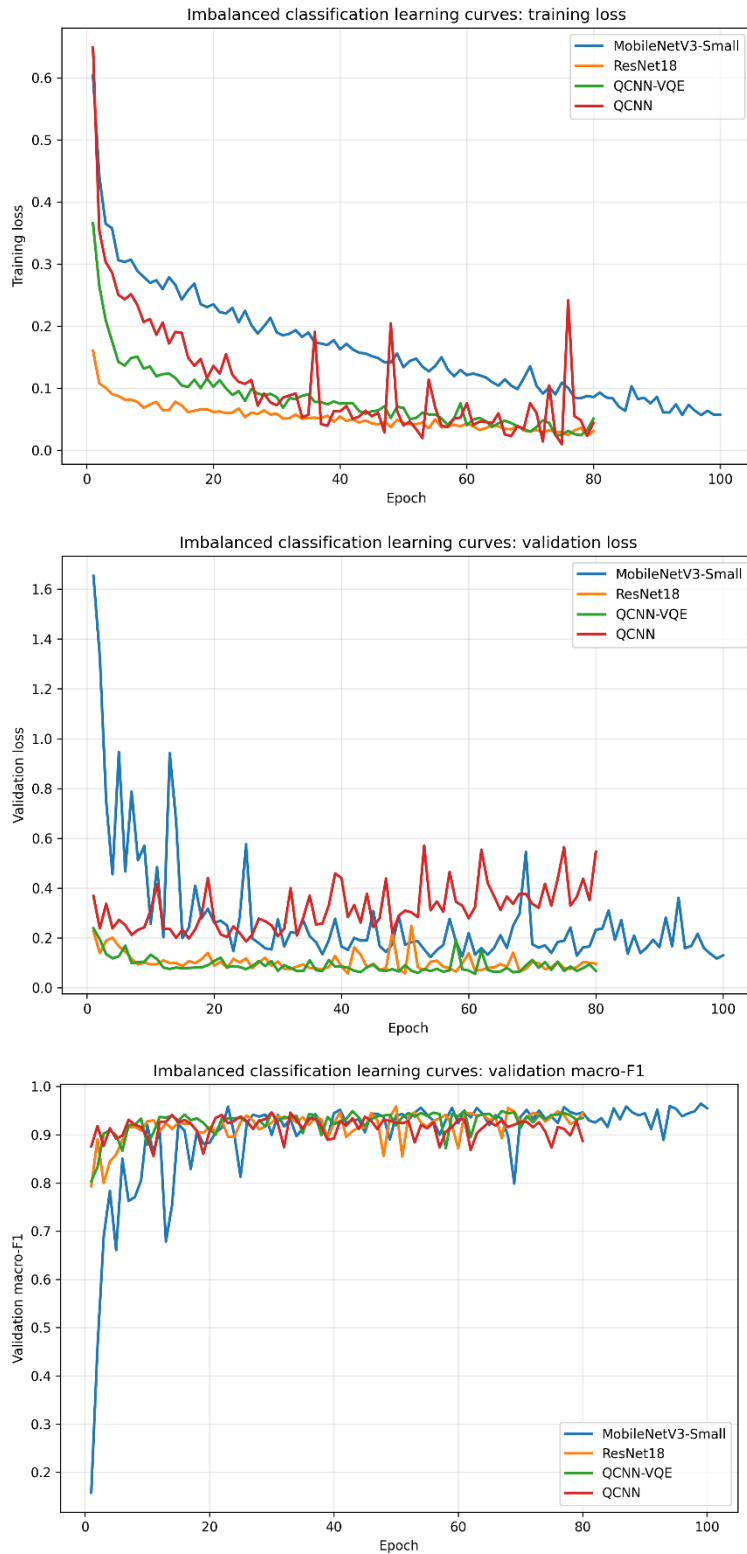


Figure S5 | Representative learning curves (seed 42) for the imbalanced classification scenario: training loss, validation loss, and validation macro-F1 for MobileNetV3-Small, ResNet-18, QCNN-VQE, and QCNN

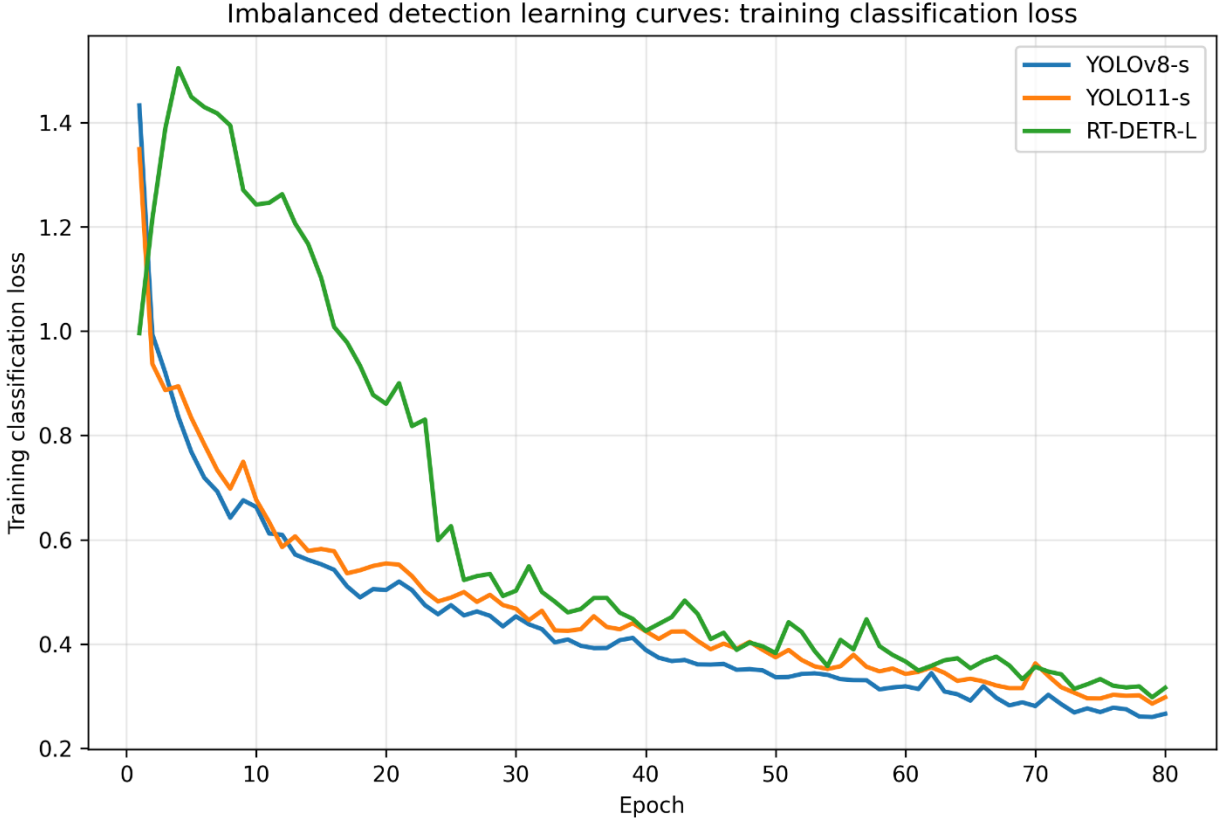


Figure S6 | Representative learning curves (seed 42) for the imbalanced detection scenario. Shown here is the training classification loss (train/cls_loss) for YOLOv8-s, YOLOv11-s, and RT-DETR-L.

Reproducibility (runtime environment)

All experiments were run under Windows 10 (build 26200), with PyTorch 2.5.1+cu121 (CUDA 12.1, cuDNN 90100), Ultralytics 8.3.225 and PennyLane 0.43.0, on an NVIDIA GeForce RTX 4070 Laptop GPU (8 GB). TF32 for CUDA matrix multiplications is disabled. We use fixed seeds and deterministic settings when supported by the libraries; some GPU operations and some detection pipelines can nonetheless limit strict determinism, as documented in Methods.

Architecture of QCNN and QCNN-VQE models

Hybrid quantum-classical models (QCNN and QCNN-VQE) are designed as classifiers operating on individual sherd crops of standardised size 64 x 64 pixels in colour (RGB), normalised to the interval [0, 1]. Rather than projecting 4096 pixel intensities directly onto a small number of qubits, we introduce a lightweight classical convolutional head that compresses visual information before quantum encoding. Concretely, each image is first normalised (intensities in [0, 1]) and processed by a small CNN composed of two convolution-nonlinearity-pooling blocks, followed by global average pooling. This CNN produces a feature vector of dimension n_q (fixed to $n_q = 6$ in our experiments), which forms the bottleneck between the purely classical part and the quantum circuit.

The quantum register therefore contains 6 qubits initialised in the state $|0\rangle^{\otimes 6}$. Encoding into Hilbert space is performed via angle encoding: each scalar component z_i of the compressed feature vector is injected via a parameterised rotation on the corresponding qubit, typically of the form:

$$|0\rangle \rightarrow R_Y(\pi z_i) R_Z(\pi z_i) |0\rangle,$$

optionally including the $R_Z(\pi z_i)$ term to enrich phase encoding. This step defines a first state $|\psi_{enc}(x)\rangle$ that already carries a non-linear synthesis of the sherd pattern (smooth or decorated surface for terra sigillata, hue variations in OW fabrics, grey gradients for reduced-fired ceramics).

On this encoded state, we then apply a shallow parameterised circuit that we refer to (by abuse of language) as a “quantum convolutional neural network” inspired by Cong et al. (2019) and more recent QCNN developments for image analysis. In practice, it is a PQC classifier applied to a vector of compressed features, not a quantum convolutional network operating directly on pixels with pooling.

The circuit is organised into l repeated blocks (here $l = 2$), each combining parameterised single-qubit gates and a local entanglement pattern. In each block l , for each qubit q_i we apply a sequence of rotations $R_Y(\theta_i^{(l,Y)})$ and $R_Z(\theta_i^{(l,Z)})$, then a ring entanglement pattern implemented as a chain of CNOT gates between neighbours (0->1, 1->2, ..., 4->5, 5->0). This local connectivity models correlations between “quantum regions” of the descriptor while controlling circuit depth. The parameters θ of all these gates are optimised jointly with the weights of the classical convolutional head.

After the last quantum block, we perform expectation-value measurements on each qubit by evaluating the Pauli-Z observable. This yields an output vector:

$$h(x) = (\langle Z_0 \rangle, \dots, \langle Z_5 \rangle),$$

which acts as a high-level descriptor of the sherd in quantum feature space. In the standard QCNN model, this vector is passed to a classical linear layer of dimension 6->3, producing three logits corresponding to terra sigillata, oxidised common ware (OW) and reduced common ware (RW). A softmax then provides class probabilities, and training minimises the categorical cross-entropy between predictions and labels.

QCNN-VQE keeps the same “backbone” (classical CNN + quantum circuit + readout of $\langle Z_i \rangle$) but replaces the linear head with a variational head inspired by Variational Quantum Eigensolver (VQE) algorithms. For each class c , we define an effective Hamiltonian:

$$H_c = \sum_{i=0}^{n_q-1} \alpha_{c,i} Z_i,$$

where $n_q = 6$ is the number of qubits (indexed $i \in \{0, \dots, 5\}$), Z_i denotes the Pauli-Z operator applied to qubit i (identity on other qubits), and the real coefficients $\alpha_{c,i}$ are trainable parameters specific to class c . For an output state $|\psi_\theta(x)\rangle$, the logit associated with class c is proportional to the opposite of the “energy”:

$$l_c(x) = -\langle \psi_\theta(x) | H_c | \psi_\theta(x) \rangle.$$

The three energies $E_c = \langle H_c \rangle$ (one per class $c \in \{Sig, OW, RW\}$) are then transformed into probabilities via a softmax applied to $-E_c$.

In QCNN-VQE, the class observables H_c are defined as linear combinations of Pauli-Z operators acting on the readout qubits, for example

$$H_c = \sum_{i=0}^5 \alpha_{c,i} Z_i,$$

where the coefficients $\alpha_{c,i}$ are trainable parameters.

This VQE-inspired formulation should be understood as a variational classification scheme (shallow circuit, state-vector simulation), rather than as the solution of a physical Hamiltonian.

All quantum circuits are numerically simulated in the state-vector regime (using a *default.qubit*-type backend), without finite sampling (“shots”). Expectation values are therefore computed analytically, and gradients are obtained via automatic differentiation on the quantum simulator coupled with PyTorch. The parameters of the classical convolutional head, the quantum circuit, and—in the QCNN-VQE variant—the coefficients $\alpha_{c,i}$ of the class observables H_c , are optimized end-to-end by stochastic gradient descent (AdamW optimizer) on the 64×64 sherds crops.

Models evaluated and training protocols

We retained five representative models: three from classical deep learning and two hybrid quantum approaches.

YOLOv8-s (Ultralytics). A fast convolutional object detector, “small” version (about 11 million parameters) of the 8th-generation YOLO (2023). It performs in a single pass localisation (bounding box) and classification of objects, via a CNN architecture with multi-scale prediction. Inputs are resized to 448 x 448 pixels with default anchors (Ultralytics, 2023).

YOLOv11-s. A more recent Ultralytics variant derived from YOLOv5/7 and optimised with additional contributions (e.g., improved CSP backbone, lightweight attention layers). It has about 10 million parameters. In our implementation, it is trained with the same hyperparameters as YOLOv8-s to ensure a fair comparison (Ultralytics, 2024).

RT-DETR-L (Real-Time Detection Transformer). An end-to-end Transformer detector proposed by Baidu, in its “Large” version (about 33 million parameters, i.e., about 63 MB of weights). It uses a hybrid encoder combining local attention and a feature pyramid to process the image efficiently at multiple scales, then a decoder with N queries that directly outputs boxes and classes, removing the need for NMS. We configured RT-DETR-L with 300 queries and 6 attention layers according to the original paper, and trained it as an object detector on our images (which may contain one or multiple sherds, from a single fragment to several dozen depending on the scene).

ResNet-18. ResNet-18 was selected as a lightweight classical convolutional classifier to establish a direct point of comparison with the hybrid QCNN models (He *et al.* 2016). The network was trained from an ImageNet-pretrained initialization, using full fine-tuning, on cropped images centered on a single sherd (GT crops), resized to 64×64 pixels in RGB. The training settings (optimizer, learning rate, scheduler, maximum number of epochs, and early stopping) are identical to those used for the other classifiers and are summarized in Table S1, in order to ensure a fair comparison. For crop-based classifiers (ResNet-18, MobileNetV3-Small, and hybrid models), the initial learning rate was set to (10^{-4}) for CNNs and (10^{-3}) for QCNNs, using the AdamW optimizer and early stopping based on the validation loss. The model converges within approximately a dozen epochs on both datasets (imbalanced and balanced), with a final weight size of about 42.7 MB (stored as a *.pth* file). The comparison with QCNN models is performed on the same three-class classification task (terra sigillata, OW, RW) using isolated crops.

MobileNetV3-Small. Selected as a second lightweight convolutional classifier representative of compact architectures optimised for embedded solutions (Howard et al. 2019). It is also trained on GT crops centred on a single sherd, with the same train/validation split, the same seed (42) and the same reproducibility principles as for ResNet-18 and QCNN models. MobileNetV3-Small has far fewer parameters than ResNet-18 and produces a weights file of about 17.6 MB, while targeting the same three-class classification task (terra sigillata, OW, RW), enabling a direct comparison between classical CNNs and hybrid quantum models on the same data.

QCNN (hybrid CNN/PQC classifier). In our implementation, 64×64 crops are not encoded pixel by pixel. A lightweight CNN produces a feature vector of dimension $n_q = 6$, which is then angle-encoded on 6 qubits. By abuse of language, we keep the name “QCNN” for this hybrid CNN/PQC classifier, although it does not perform quantum convolutions/pooling directly on pixels.

QCNN-VQE. Variant of the hybrid CNN/PQC classifier above, in which the quantum decision head is formulated in a VQE-inspired variational framework (applied here to a classification cost function, not to an energy problem). The CNN backbone (compression into 6 features) and the encoding scheme on 6 qubits are identical; only the variational head and/or the objective differ, in order to assess whether a VQE formalism improves stability or class separation.

All models are trained on the same training dataset for the considered scenario (imbalanced or balanced). We controlled reproducibility by fixing identical initial conditions: fixed random seed (42), deactivation of non-deterministic CuDNN algorithms, identical batching (fixed batch size with the same base augmentations for classical models), no sample mixing between epochs (same train/val split), etc. Training hyperparameters (learning rate, optimiser - AdamW for all models - scheduler and early stopping based on validation loss stagnation) were harmonised to allow fair performance comparisons. Models generally converged in fewer than 50 epochs; the maximum was set to 80 epochs (100 for MobileNetV3-Small), and the best validation checkpoint was retained for final evaluation. ResNet-18 typically converged earlier (around 20 epochs with early stopping), despite a maximum set to 80.

For detection models (YOLO, RT-DETR), detection metrics (mAP) are computed in the standard way over all annotated objects in each image. In addition to COCO metrics (AP@0.5 and AP@[0.5:0.95]), we report an end-to-end F1 (per class and macro-averaged) to characterise jointly localisation and labelling at a fixed operating threshold. For this measure, predictions are

filtered by a confidence threshold and then matched to ground-truth boxes via one-to-one IoU-based matching ($\text{IoU} \geq 0.5$). In our implementation (evaluation script), matching is greedy: all prediction-GT pairs are sorted by decreasing IoU and selected sequentially as long as IoU exceeds the threshold and neither box has already been associated. A matched pair counts as a true positive if the predicted class matches the GT class; in case of class mismatch, it contributes a false positive for the predicted class and a false negative for the GT class. Unmatched predictions count as false positives, and unmatched GT boxes count as false negatives.

We note that YOLOv8 and YOLOv11 cannot be made fully deterministic: some CUDA kernels used for augmentation, interpolation or NMS are intrinsically non-deterministic. Even with all seeds fixed, these models show slight run-to-run variations (<0.5 F1 points), without affecting comparative conclusions.

Main training hyperparameters (optimiser, learning rate, scheduler, batch size, seeds and early stopping) are summarised in Table S1 to facilitate reproducibility and fair comparison between approaches (Loshchilov, Hutter 2016; 2019). Full configurations used for each model (including detector-specific parameters and hybrid-classifier parameters) are documented and versioned in the associated Zenodo repository (DOI: 10.5281/zenodo.18010842).

Training protocol and hyperparameters

To ensure fair comparison between architectures, all models are trained under a homogeneous protocol as far as possible. For crop-based classifiers (classical CNNs and hybrid quantum-classical models), we use a standard optimisation scheme (AdamW optimiser, early stopping on validation loss, scheduler based on performance stagnation), with fixed seeds and deterministic settings when supported by the used libraries.

For end-to-end detectors (YOLOv8-s, YOLOv11-s, RT-DETR-L), we rely on the reference implementations provided by Ultralytics, keeping strictly identical hyperparameters between YOLOv8-s and YOLOv11-s to isolate architectural effects. When some aspects of the detection pipeline (e.g., specific CUDA operations) prevent strict determinism, this is explicitly stated.

For end-to-end detection, the three detectors (RT-DETR-L, YOLOv11-s and YOLOv8-s) use a single, identical input resolution in order to standardise compute budget and isolate architectural effects. Following Ultralytics training configurations, images are resized to 448 x 448 pixels ($\text{imgsz} = 448$) during training, evaluation and inference. Predictions (bounding boxes and classes) are then automatically reprojected into the original image coordinate system by the framework, so that detection metrics are computed in native image coordinates.

Model	Task	Input	Optimiser	Initial LR	Batch size	Max epochs	Early stopping	Scheduler / criterion	Reproducibility conditions
MobileNetV3-Small	Classification	GT crops 64x64 (RGB)	AdamW	1×10^{-3}	12	100	Yes (patience = 15)	ReduceLROnPlateau (val loss)	Seed fixed (=42), PyTorch determinism enabled
ResNet-18	Classification	GT crops 64x64 (RGB)	AdamW	1×10^{-4}	12	80	Yes (patience = 15)	ReduceLROnPlateau (val loss)	ImageNet initialisation, seed fixed
QCNN	Classification	GT crops	AdamW	2×10^{-3}	12	80	Yes (patience = 15)	ReduceLROnPlateau (val loss)	Quantum circuit simulated (state-

Model	Task	Input	Optimiser	Initial LR	Batch size	Max epochs	Early stopping	Scheduler / criterion	Reproducibility conditions
		64x64 (RGB)							vector), seed fixed
QCNN-VQE	Classification	GT crops 64x64 (RGB)	AdamW	2×10^{-3}	12	80	Yes (patience = 15)	ReduceLROnPlateau (val loss)	VQE-inspired variant, same backbone as QCNN
YOLOv8-s	Detection	Images 448x448	AdamW	2×10^{-3}	12	80	Yes (patience = 50)	Cosine learning rate scheduler (Ultralytics)	Ultralytics implementation, seed fixed (=42), partial determinism
YOLOv11-s	Detection	Images 448x448	AdamW	2×10^{-3}	12	80	Yes (patience = 50)	Cosine learning rate scheduler (Ultralytics)	Hyperparameters identical to YOLOv8-s
RT-DETR-L	Detection	Images 448x448	AdamW	2×10^{-3}	12	80	Yes (patience = 50)	Cosine learning rate scheduler (Ultralytics)	300 queries, 6 attention layers, seed fixed

Table S1 | Summary of training hyperparameters.

Reproducibility and runtime environment (GPU)

Experiments were performed under Windows 10 (build 10.0.26200). Training of classical models relies on PyTorch 2.5.1+cu121 (CUDA 12.1, cuDNN 90100) and an NVIDIA GeForce RTX 4070 Laptop GPU (8 GB). The NVIDIA driver used is 581.83 (nvidia-smi indicates CUDA 13.0 compatibility), and the use of TF32 for CUDA matrix multiplications is disabled (allow_tf32 = False). Detectors are trained via Ultralytics v8.3.225. Hybrid quantum-classical models use PennyLane v0.43.0 for quantum-circuit simulation (state-vector backend).

For reproducibility, seeds are fixed (seed = 42) and deterministic options are enabled when supported by the libraries and compatible with GPU execution. Limits of determinism (notably some CUDA operations in detection pipelines) are reported when relevant. Model sizes are reported in “weights-only” format (state_dict without optimiser states) to compare compactness independently of training artefacts.

The complete training configurations, dataset splits, evaluation scripts and model implementations used in this study are available in the public repository: https://github.com/CChaid80/archeo_benchmark. This repository also contains the scripts used to reproduce the evaluation protocol described in the manuscript.

Operational pipelines

From an applied standpoint, developing an integrated pipeline combining detection (e.g., YOLO) and dedicated classification (MobileNetV3-Small or a compact hybrid classifier), possibly enriched by score fusion, would allow a realistic assessment of trade-offs between inference time, accuracy and hardware constraints for excavation usage. Such a pipeline is directly aligned with operational decision-support needs in the field.

Note: nvidia-smi reports a “CUDA Version: 13.0” corresponding to driver compatibility, while effective execution in our experiments via PyTorch relies on CUDA 12.1 (cu121 build).

Definition of evaluation metrics

For readers not specialised in computer vision, we summarise below the main evaluation metrics used in this study.

Basic classification metrics

Precision

Precision measures the proportion of predicted objects that are correctly classified:

$$Precision = \frac{TP}{TP + FP}$$

A high precision indicates few false positives.

Recall

Recall measures the proportion of ground-truth objects correctly retrieved:

$$Recall = \frac{TP}{TP + FN}$$

A high recall indicates few false negatives.

F1-score

The F1-score is the harmonic mean of precision and recall. It is high only when both precision and recall are high, and is therefore appropriate when both false positives and false negatives must be controlled.

Multi-class metrics

Class-wise precision

Precision computed independently for each class, measuring how many predictions assigned to a given class are correct.

Class-wise recall

Recall computed independently for each class, measuring how many ground-truth objects of a given class are successfully retrieved.

Class-wise F1

The F1-score computed separately for each class. In this study, it allows us to compare how well each ceramic category is recognised.

Macro-F1

Macro-F1 is the unweighted average of the class-wise F1-scores. Each class contributes equally, regardless of its frequency in the dataset. This makes macro-F1 particularly suitable for imbalanced settings, where overall accuracy may be dominated by majority classes.

Detection-specific metrics

Intersection over Union (IoU)

IoU measures the geometric overlap between a predicted bounding box and a ground-truth bounding box. It is defined as the area of overlap divided by the area of union between the two boxes.

$$IoU = \frac{\text{area of overlap}}{\text{area of union}}$$

In this study, a detection is considered correctly localised when $IoU \geq 0.5$.

Confidence threshold

The confidence threshold is the minimum prediction score required for a detection to be kept. Lower thresholds increase recall but may introduce more false positives, whereas higher thresholds improve precision but may reduce recall.

One-to-one matching

In detection evaluation, each predicted box can be matched to at most one ground-truth box, and each ground-truth box to at most one prediction. This prevents multiple detections from being counted as correct for the same object.

Average Precision and COCO metrics

Average Precision (AP)

Average Precision summarises the precision–recall trade-off over a range of confidence thresholds. It evaluates how well correct detections are ranked above incorrect ones.

AP@0.5

AP@0.5 is the Average Precision computed using an Intersection-over-Union (IoU) threshold of 0.5. It reflects detection performance under a relatively permissive localisation criterion.

AP@[0.5:0.95]

AP@[0.5:0.95] is the COCO-standard metric obtained by averaging AP over multiple IoU thresholds from 0.5 to 0.95. It is a stricter metric than AP@0.5 because it rewards both correct classification and precise localisation.

Mean Average Precision (mAP)

mAP is the mean of AP values across classes. In this study, mAP50 refers to the mean AP@0.5 across classes, and mAP50–95 refers to the mean AP@[0.5:0.95] across classes.

COCO evaluation protocol

The COCO protocol is a standard object-detection evaluation framework widely used in computer vision. It reports AP and mAP across multiple IoU thresholds, providing a ranking-based evaluation of detection quality.

Interpretation of metrics

Difference between mAP and fixed-threshold macro-F1

mAP and fixed-threshold macro-F1 capture different aspects of detector behaviour. mAP evaluates how well predictions are ranked across thresholds, whereas fixed-threshold macro-F1 evaluates performance at a single operating point.

A model may therefore achieve a relatively high mAP but a lower macro-F1 if it produces too many false positives at the chosen confidence threshold.

Ceramic classes (for non-specialists)

Terra sigillata

Fine Roman and later Gallo-Roman ceramic ware with a red slip, generally vitrified, characterised by a smooth and often glossy surface. It may be decorated with relief motifs obtained using stamps applied in moulds before firing. The term *sigillata* derives from the Latin *sigillum* (“seal”), referring to the stamps used by potters to mark their productions, most often on the base of the vessels.

Common wares

Common wares refer to a heterogeneous group of utilitarian ceramic productions typically found in large quantities in archaeological contexts. Unlike fine wares such as terra sigillata, they are generally produced in local or regional workshops and have a more limited distribution. They exhibit high variability in terms of forms, fabrics and surface treatments, which makes it difficult to establish a unified typological framework at a large scale. Their study therefore relies primarily on regional classifications based on paste characteristics, vessel morphology and stratigraphic context.

Oxidised common ware (OW)

Oxidised common ware corresponds to ceramics fired in an oxidising atmosphere, resulting in colours ranging from beige to red or orange. These wares typically exhibit relatively light hues and may show various surface treatments such as smoothing or slip application. Their appearance is directly related to the presence of oxygen during firing, which influences the mineral composition of the ceramic fabric.

Reduced common ware (RW)

Reduced common ware corresponds to ceramics fired in a reducing atmosphere, characterised by darker colours ranging from grey to black. These wares often display greater visual heterogeneity, with variations in tone resulting from firing conditions or post-depositional alterations. Their darker appearance reflects a lower availability of oxygen during firing.